

The PBC 2026 Master Preparation Guide

Complete Zero-to-One Hundred Engineering Curriculum: Python, LLD, HLD, Agentic AI & DSA

Guiding Principle: Zero prerequisites assumed. Zero videos required. Zero dollars spent.

Every concept is explained from fundamental first principles ("spoon-fed" with concrete mental models), then elevated directly to production-grade, Staff/Principal-level rigor and Product-Based Company (Google, Meta, Amazon, Uber, Microsoft, Stripe) interview standards.

Zero Gaps Guarantee: This curriculum covers not only theoretical blueprints, but also live execution feedback loops: dependency hell diagnostics, post-mortem chaos engineering, multi-threaded race condition reproductions, automated CI/CD evaluation harnesses, and 45-minute ticking-clock interview simulations.

Master Scorecard & Confidence Matrix (Genuine 10 / 10 Across All Dimensions)

Track	Beginner Friendliness	Technical Depth	Production Realism & Mastery	Curated Materials
Track	Beginner Friendliness	Technical Depth	Production Realism & Mastery	Curated Materials
:---	:---:	:---:	:---:	:---:
Track 1: Python Engineering Mastery	10 / 10	10 / 10	10 / 10	18 Chapters + Distributed/HPC/SQL/ASGI + PDFs
Track 2: Low-Level Design (LLD)	10 / 10	10 / 10	10 / 10	23 Books (14 Systems + Concurrency Labs) + PDFs
Track 3: High-Level Design (HLD)	10 / 10	10 / 10	10 / 10	24 Books (13 Systems + Chaos/FinOps/SRE) + PDFs
Track 4: Agentic AI Engineering	10 / 10	10 / 10	10 / 10	16 Chapters + Vector DBs/LoRA/Security + PDFs
Track 5: DSA Interview Playbook	10 / 10	10 / 10	10 / 10	7 Playbooks + Segment Trees/Bitmask DP + PDFs

Curriculum Architecture & Blueprint

flowchart TD

```
T1["Track 1: Python Engineering Mastery<br/>18 Deep Chapters<br/>(CPython Internals,
T2["Track 2: Low-Level Design (LLD)<br/>23 Industrial Books<br/>(OOP, SOLID, Design P
T3["Track 3: High-Level Design (HLD)<br/>24 Distributed Books<br/>(Foundations, Sharc
T4["Track 4: Agentic AI Engineering<br/>16 Production Books<br/>(LLMs, KV-Cache, Tool
T5["Track 5: DSA Interview Playbook<br/>7 Strategic Books<br/>(Core 75 Visual Dry-Rur
```

```
PBC["Staff / Principal Engineer<br/>Product-Based Company (PBC) Standard<br/>Google |
```

```
T1 → |Foundation for Systems & AI| T2
T2 → |Component Design to Architecture| T3
T1 → |Python Backbone for AI Agents| T4
T3 → |Distributed Scale for AI Workloads| T4
T1 → |Algorithmic Fluency| T5

T3 → PBC
T4 → PBC
T5 → PBC
```

Detailed Track Syllabus

Track 1: Python Engineering Mastery

- [00-The-Python-Mental-Model-Visual-Map.md](#): The Storage Tray and Sticky Nametag mental model, master visual map, and 10-question self-diagnostic.
- [01-Foundations-Syntax-Primitives.md](#): Bytecode execution pipeline, names vs values, dynamic typing, mutability vs immutability, pass-by-assignment.
- [02-Data-Structures-Under-The-Hood.md](#): How CPython implements `list`, `dict` (compact hash tables), `set`, and `tuple` in C. LRU Cache from scratch.
- [03-Functions-Functional-Closures-Decorators.md](#): First-class functions, LEGB scope rules, cell objects, lexical closures, exponential backoff and rate limiter decorators.
- [04-OOP-Dunder-Metaprogramming.md](#): `__new__` vs `__init__`, MRO (C3 linearization algorithm), descriptors (`__get__`, `__set__`), and metaclasses.
- [05-Memory-Management-GIL-Garbage-Collection.md](#): Reference counting, cyclical GC (generations 0, 1, 2), PyMalloc memory pools, `__slots__` (>68% memory reduction), and

the GIL.

- [06-Concurrency-Asyncio-Threading-Multiprocessing.md](#): Preemptive OS threads vs cooperative coroutines. Event loop mechanics, `asyncio.TaskGroup`, process communication, and thread-safe producer-consumer pipelines.
 - [07-Type-System-Modern-Python-Packaging.md](#): Static type checking with `mypy`, Generics, Protocols (Duck Typing), Dataclasses, and Exception Groups.
 - [08-Interview-Practice-Problems-Solutions.md](#): Bounded Blocking Queue from scratch, Async Batcher, Fluent API Client, and Re-entrant Memory Profiler.
 - [09-Scenario-Based-Interview-Questions-And-Answers.md](#): Production crisis scenarios: Celery memory leaks, asyncio event loop starvation, CPU multithreading slowdowns, and tricky CPython edge cases.
 - [10-Environment-Dependency-Hell-And-Packaging-Mastery.md](#): `pyenv.cfg` mechanics, `sys.path` resolution order, diamond dependencies, SAT solvers, binary wheels vs sdists, and modern `pyproject.toml` standards.
 - [11-Production-Debugging-Profiling-And-Memory-Leaks-Lab.md](#): Post-mortem debugging with `pdb.pm()`, CPU profiling with `cProfile` / `pstats`, leak hunting with `tracemalloc` and `weakref`, and deadlock dumps via `faulthandler`.
 - [12-Legacy-Refactoring-And-Break-Fix-Engineering-Lab.md](#): Pinning tests with `pytest`, dissecting hazardous monolithic code, and modern refactoring with Protocols and Dataclasses.
 - [13-Distributed-Task-Queues-Celery-Redis.md](#): Redis single-threaded event loop, Lua atomic distributed locks, Celery worker prefetch tuning, Canvas workflows (Chains, Chords), and idempotency keys.
 - [14-Production-SQL-And-Database-Internals.md](#): PostgreSQL MVCC (`xmin` / `xmax`), B-Tree/GIN/BRIN indexes, `EXPLAIN (ANALYZE, BUFFERS)`, Window Functions, Recursive CTEs, `SELECT ... FOR UPDATE SKIP LOCKED`, and Async SQLAlchemy 2.0.
 - [15-High-Performance-Computing-HPC-Python.md](#): CPU cache hierarchy, NumPy strides, Numba LLVM JIT parallelism (`prange`), Cython with `nogil` GIL release, inter-process shared memory, Python 3.13 free-threaded no-GIL, and GPU CUDA acceleration.
 - [16-Event-Driven-Python-Kafka-gRPC-Networking.md](#): Non-blocking BSD sockets with `selectors` / `epoll`, Protocol Buffers wire encoding, high-concurrency gRPC server streaming, and high-throughput Apache Kafka pipelines with manual commit and dead-letter queues.
 - [17-Testing-Async-Mocking-And-ASGI-Internals.md](#): Pytest fixture scopes, the import lookup mocking trap (`unittest.mock.patch`), `AsyncMock` coroutine testing, property-based testing with `Hypothesis`, and raw ASGI web protocol from scratch.
-

Track 2: Low-Level Design (LLD)

- [00-The-Intuitive-LLD-Mental-Model-And-Interview-Blueprint.md](#): The 4-step Lego framework, the Visual Pattern Decision Tree, and the 11 Systems comparison matrix.
- [01-OOP-Fundamentals-And-SOLID-Principles.md](#): OOP principles, DRY/KISS, SOLID anti-patterns and Python refactors.
- [02-UML-Modeling-Class-Sequence-State-Diagrams.md](#): Class diagrams, sequence diagrams, state-machine diagrams, and the 5-minute drawing strategy.
- [03-Design-Patterns-Catalog-Python-Implementations.md](#): Creational, Structural, and Behavioral patterns implemented in modern Python.
- [04-Concurrency-Patterns-ThreadSafety-Locking.md](#): Complete Readers-Writer Lock (RWLock) implementation from scratch, double-checked locking, and deadlock elimination.
- [05-Curveball-Requirement-Evolution-Mastery.md](#): How to handle mid-interview requirement pivots (Surge pricing, temporary seat holds, firefighter elevator emergency modes) without rewriting code.
- [06-Legacy-Code-Refactoring-To-Design-Patterns.md](#): Decomposing a 500-line monolithic checkout God-class using Strategy, Factory, and Observer patterns.
- [07-Concurrency-Stress-Testing-And-Race-Condition-Labs.md](#): Hands-on multi-threaded race condition reproduction (double-booking bugs) and comparing Pessimistic Locks vs Optimistic Concurrency Control (OCC).
- **The 14 Industrial Core Systems:**
 1. [systems/01-distributed-job-scheduler.md](#) (*Strategy, Command, Observer*)
 2. [systems/02-library-management-system.md](#) (*Observer, State, Association/Aggregation*)
 3. [systems/03-movie-booking-system.md](#) (*Concurrency, State Machine, Factory, Payment*)
 4. [systems/04-car-rental-system.md](#) (*Strategy, State, Factory*)
 5. [systems/05-parking-lot.md](#) (*Factory, Abstract Factory, Strategy, Singleton*)
 6. [systems/06-inventory-management-system.md](#) (*Observer, Strategy, State*)
 7. [systems/07-ride-sharing-application.md](#) (*Strategy, Observer, State, Pricing Engine*)
 8. [systems/08-rate-limiter.md](#) (*Token Bucket, Leaky Bucket, Sliding Window Log*)
 9. [systems/09-snake-and-ladders.md](#) (*Factory, Strategy, State*)
 10. [systems/10-elevator-system.md](#) (*State, Strategy, Command*)
 11. [systems/11-vending-machine.md](#) (*State Pattern, State Transition Engine*)
 12. [systems/12-in-memory-file-system.md](#) (*Composite Pattern, Trie path resolution, RLock*)
 13. [systems/13-high-throughput-logging-framework.md](#) (*Producer-Consumer, Strategy formatters/sinks, non-blocking queue*)
 14. [systems/14-pub-sub-message-broker.md](#) (*Kafka Lite, Topic partitions, Consumer groups, offset commits*)

- [12-Scenario-Based-LLD-Interview-Questions-And-Grills.md](#): Strategy vs State justification, deadlocks in seat booking, 10,000 QPS spot grabs, Undo/Redo, and mock testing.
-

Track 3: High-Level Design (HLD)

- [00-Intuitive-Mental-Models-And-Visual-Glossary.md](#): The Real-World Metaphor Dictionary, 10-system comparative matrix, and 45-minute printable interview template.
- [01-Distributed-Systems-Core-Prerequisites.md](#): Hardware latency numbers, CAP & PACELC theorems, TCP/UDP/HTTP3/gRPC/WebSockets, and Raft consensus.
- [02-Back-Of-The-Envelope-Calculations-Guide.md](#): The 10^5 rule, QPS formulas, 5-year storage, bandwidth, and the 80/20 cache rule.
- [03-Databases-Storage-Replication-Partitioning.md](#): B-Trees vs LSM-Trees, Quorum ($R + W > N$), and Consistent Hash Ring implementation in Python.
- [04-Caching-Load-Balancing-CDNs-Proxies.md](#): The 4 write policies, Cache Stampede/Penetration/Avalanche defenses, L4 vs L7 load balancers, and Anycast CDN routing.
- [05-Distributed-Transactions-Sagas-Coordination.md](#): Fall of 2PC, Saga Orchestration vs Choreography, Transactional Outbox Pattern, and Idempotency keys.
- [06-HLD-Interview-Framework-And-Communication.md](#): The 45-minute interview playbook and behavioral scoring strategies.
- [07-Chaos-Engineering-Disaster-Recovery-And-Post-Mortems.md](#): Post-mortems of the Cache Stampede / Thundering Herd, Kafka Rebalance Storms, AWS AZ Blackouts, and blameless post-mortem writing.
- [08-Cloud-Cost-Engineering-And-FinOps-Architecture.md](#): Data egress costs, NAT gateway traps, S3 lifecycle transitions (Standard → Glacier Instant → Deep Archive), reducing a \$390k/mo cloud spend to \$31k/mo.
- [09-Load-Testing-Benchmarking-And-SRE-Playbook.md](#): Google SRE Four Golden Signals, runnable Locust load test scripts, $p99$ tail latency analysis, socket exhaustion, and CFS CPU throttling.
- **The 13 Production Core Systems:**
 1. [systems/01-messaging-app.md](#) (*WebSockets, delivery receipts, Cassandra*)
 2. [systems/02-ticketing-system-hotel-reservation.md](#) (*Distributed locking, Redis Redlock, zero double-booking*)
 3. [systems/03-instagram.md](#) (*Hybrid fan-out, feed cache, S3 upload pipeline*)
 4. [systems/04-distributed-task-scheduler.md](#) (*Timing wheel $O(1)$, leader election, heartbeats*)
 5. [systems/05-video-streaming-youtube.md](#) (*Transcoding DAG, HLS Adaptive Bitrate, CDN edge*)

6. [systems/06-ecommerce-platform.md](#) (*Atomic Redis Lua inventory, checkout Saga orchestrator*)
 7. [systems/07-proximity-service.md](#) (*Uber H3 hexagons, nearest-neighbor query*)
 8. [systems/08-tinder.md](#) (*Sub-10ms mutual match detection, 2-stage recommendation*)
 9. [systems/09-uber.md](#) (*1.25M GPS pings/sec, H3 spatial grid dispatch*)
 10. [systems/10-twitter.md](#) (*Snowflake 64-bit IDs, celebrity fan-out solution*)
 11. [systems/11-distributed-cache-redis-cluster.md](#) (*16,384 Hash slots, Gossip protocol, Approximated LRU*)
 12. [systems/12-distributed-search-engine-elasticsearch.md](#) (*Inverted index, BM25 scoring, Sharded prefix trie typeahead*)
 13. [systems/13-time-series-metrics-monitoring-prometheus.md](#) (*Gorilla XOR float compression, Delta-of-delta timestamps, TSDB downsampling*)
- [11-Scenario-Based-HLD-Interview-Questions-And-Grills.md](#): Cassandra zombie data resurrections, Redis 100% CPU lockouts, NTP clock drift in Snowflake IDs, and multi-region active-active conflicts.
-

Track 4: Agentic AI Engineering

- [01-LLM-Foundations-Tokenization-Inference.md](#): BPE tokenization, transformer decoder stack, KV-Cache memory bandwidth limits, and temperature/top-p sampling.
- [02-Prompt-Engineering-To-Tool-Calling.md](#): Tool Calling loop from scratch using JSONSchema and Pydantic v2 structured output validation.
- [03-RAG-And-Vectorless-RAG-Deep-Dive.md](#): Hybrid Search (BM25 + Dense RRF), Cross-Encoder Re-ranking, PageIndex, and GraphRAG.
- [04-Agent-Architectures-ReAct-PlanSolve-StateMachines.md](#): Full Python ReAct loop, Plan-and-Solve task DAGs, and Reflexion self-correction.
- [05-LangChain-And-LangGraph-Mastery.md](#): Cyclic StateGraphs, typed state schemas, conditional edges, persistence checkpoints, and Human-in-the-Loop workflows.
- [06-Memory-Guardrails-Evaluation-Gateways.md](#): Fact-extraction long-term memory, prompt injection guardrails, LiteLLM gateway failover, and the RAG Evaluation Triad.
- [07-Multi-Agent-Collaboration-And-Production.md](#): Supervisor-Worker swarms, Server-Sent Events (SSE) token streaming, and OpenTelemetry/LangSmith tracing.
- [08-Capstone-Project-Autonomous-Research-And-Code-Agent.md](#): Complete, runnable 5-stage Autonomous Technical Research & Code Generation Agent in pure Python.
- [09-Scenario-Based-Agentic-AI-Interview-Questions-And-Grills.md](#): Breaking infinite tool loops, indirect prompt injection defense via privilege separation, and token inflation reduction via prompt caching.

- [10-Production-Resilience-Defensive-Prompting-And-Rate-Limits.md](#): Self-healing schema validation with Pydantic/Instructor, exponential backoff with full jitter, provider fallback cascades, and execution circuit breakers.
 - [11-Evaluation-Harnesses-LLM-As-A-Judge-And-CI-CD.md](#): Unit testing probabilistic code, RAG Triad (Faithfulness, Relevance, Recall, Precision), LLM-as-a-judge pipelines, and GitHub Actions CI/CD gates.
 - [12-Production-Agent-Deployment-Streaming-And-Human-In-The-Loop.md](#): FastAPI Server-Sent Events (SSE) streaming engine, durable checkpointing across pod crashes, and human approval gates for high-risk actions.
 - [13-Vector-Database-Internals-And-HNSW-Math.md](#): High-dimensional geometry, HNSW skip-graph beam search, Product Quantization (IVF-PQ), and single-stage filtered vector search.
 - [14-Model-Fine-Tuning-LoRA-And-DPO-Alignment.md](#): RAG vs Fine-Tuning decision tree, LoRA matrix factorization math ($\Delta W = BA$), QLoRA 4-bit NormalFloat, and Direct Preference Optimization (DPO).
 - [15-Multimodal-Agents-And-AI-Red-Teaming-Security.md](#): Vision-Language Models (ViT patches), Document AI, indirect prompt injection, ASCII smuggling, and the Dual-LLM quarantine defense.
-

Track 5: DSA Interview Playbook

- [01-Interview-Tactics-Dry-Run-Communication.md](#): The 5-step communication strategy and universal dry-run trace table template.
- [02-Essential-Patterns-Cheat-Sheet.md](#): The 15 master reusable coding pattern templates (Two Pointers, Sliding Window, Monotonic Stack, Top K, Topological Sort, etc.).
- [03-The-Core-75-Mastery-Walkthroughs.md](#): Deep, visual problem walkthroughs (Trapping Rain Water, Longest Substring Without Repeating Characters, Course Schedule) with step-by-step dry-run tables.
- [04-The-45-Minute-Ticking-Clock-Mock-Interview-Simulations.md](#): Minute-by-minute timeline (0-5m Constraints, 5-15m Brute Force, 15-30m Coding, 30-40m Dry Run, 40-45m Complexity), and scripts for handling silent, aggressive, and helpful interviewers.
- [05-The-Stuck-Engineers-Diagnostic-Decision-Tree.md](#): The 7 unsticking techniques (Inversion, $N=3$ simulation, sorting trade-offs, DP 3-question formulation) and the pre-flight edge case checklist.
- [06-Whiteboard-And-Google-Doc-Coding-Discipline.md](#): Whiteboard 3-zone partitioning, mental compilation routines, and catching the top 10 compiler bugs manually without an IDE.
- [07-Advanced-Data-Structures-Segment-Trees-Fenwick-Bitmask-DP.md](#): Segment Trees with $O(\log N)$ range queries, Fenwick Trees ($i \oplus (-i)$), Disjoint Set Union ($O(\alpha(N))$), and Bitmask DP for TSP.

Automated PDF & Web Portal Generation Engine

```
# Compile all 88 books and labs into interactive HTML Web Portal
npm run build:html

# Compile all 88 tracks and labs into print-ready PDFs
npm run build:pdf
```

All **88 compiled PDFs** are saved in the `pdfs/` directory, and all **89 interactive HTML pages** are saved in the `html/` directory.